

RAPPORT DE STAGE ET DE RÉALISATION TECHNIQUE

Développement de la Couche d'Ingestion Intelligente de CV (PDF / DOCX / OCR)

Explication du Code Source et Préparation aux Questions de Soutenance

Entreprise d'Accueil : ARTIWEB (Fès)

I. PRÉSENTATION DE L'ENTREPRISE D'ACCUEIL : ARTIWEB

1.1 Présentation Générale

- **Nom de l'entreprise** : ARTIWEB (ARTI web)
- **Site Web Officiel** : <https://artiweb.ma/>
- **Siège Social** : Fès, Maroc
- **Partenariats** : Partenaire Google officiel (Google Partner)
- **Horaires** : Lundi – Vendredi (9:00 – 21:00)
- **Contact** : (+212) 664-017447 | contact@artiweb.ma

1.2 Secteurs d'Activité et Expertises

ArtiWeb est une agence de communication et de marketing digital basée à Fès. Elle accompagne les entreprises dans la transformation digitale et l'optimisation de leur présence en ligne à travers plusieurs pôles d'expertise :

- **Stratégie Digitale & Marketing Numérique** : Conduite de campagnes de communication, branding et visibilité digitale.
- **Développement Web & Applications** : Conception de sites web sur-mesure, plateformes e-commerce et applications métiers.
- **Référencement & Acquisition** : Positionnement SEO sur moteurs de recherche, gestion de campagnes Google Ads et Social Ads.
- **Studio Marketing & Marketplace** : Création visuelle, contenu multimédia et optimisation des canaux de vente e-commerce.

II. FICHE PROJET : AGENT IA RH (RECRUTIA)

2.1 Intitulé du Projet

Agent IA RH — Système intelligent d'automatisation du processus de recrutement (Nom court : RecrutIA).

2.2 Problématique

Le recrutement traditionnel repose sur le tri manuel de chaque CV. Cette méthode devient un goulot d'étranglement avec un volume élevé de candidatures : elle est chronophage, sujette aux erreurs humaines et manque de traçabilité.

Problématique centrale : Comment automatiser l'analyse et le tri des candidatures pour réduire la charge de travail des RH, tout en garantissant que les décisions restent fiables, explicables et sous contrôle humain sur les cas sensibles ?

2.3 Architecture en 3 Couches

- **Couche 1 - Ingestion (Tâche réalisée)** : Extrait et structure automatiquement les données des CV bruts (Texte, OCR, Compétences, Expérience, Formation) sans dépendre d'un LLM.
- **Couche 2 - IA Agentique** : Compare sémantiquement chaque profil à l'offre d'emploi, calcule un score de correspondance et génère une justification argumentée.
- **Couche 3 - Supervision Humaine** : Le recruteur garde la décision finale (validation, correction ou rejet) sur chaque préconisation de l'agent (Human-in-the-loop).

III. EXPLICATION DÉTAILLÉE DE LA TÂCHE & DU CODE SOURCE

Le module d'ingestion transforme un fichier CV brut (.pdf ou .docx) en un dictionnaire JSON structuré en 4 étapes successives.

3.1 Étape 1 : Extraction du texte en cascade (ingestion/extractor.py)

Pourquoi une stratégie en cascade ? Un CV peut être un PDF textuel natif, un Word DOCX, ou une image/scan sans texte sélectionnable. Le code tente les méthodes rapides en premier, puis passe à l'OCR en dernier recours.

[CODE] Code source exact de extractor.py :

```
def extraire_texte(chemin_fichier: str) -> str:
    chemin = Path(chemin_fichier).resolve()
    if not chemin.exists():
        raise FileNotFoundError(f"Fichier introuvable : {chemin}")

    extension = chemin.suffix.lower()

    # 1. Si fichier Word DOCX
    if extension == ".docx":
        return _extraire_docx(str(chemin))

    # 2. Si fichier PDF : Cascade d'extraction
    elif extension == ".pdf":
        # Tentative 1 : pdfplumber (précis sur texte natif)
        texte = _extraire_pdf_pdfplumber(str(chemin))
        if len(texte.strip()) >= 80: # Seuil minimal
            return texte

        # Tentative 2 : PyMuPDF / fitz (rapide)
        texte = _extraire_pdf_pymupdf(str(chemin))
        if len(texte.strip()) >= 80:
```

```

        return texte

    # Tentative 3 : OCR Tesseract (PDF scanné / image)
    texte = _extraire_pdf_ocr(str(chemin))
    return texte
else:
    raise ValueError(f"Format non supporté : '{extension}'")

```

[EXPLICATION] Explication pour la soutenance (Questions du professeur) :

- **Gestion des scans** : Si le professeur demande 'Comment gérez-vous un CV scanné ?', répondez que la fonction `_extraire_pdf_ocr()` utilise PyMuPDF pour convertir la page PDF en image PNG haute définition (300 DPI), puis passe cette image à Tesseract OCR qui lit le texte pixel par pixel.
- **Seuil de 80 caractères** : Le seuil de 80 caractères permet de détecter si l'extraction native a échoué (un PDF scanné renvoie 0 à 20 caractères d'en-tête).

3.2 Étape 2 : Nettoyage et Normalisation (ingestion/cleaner.py)

Le texte brut extrait contient souvent du bruit : puces décoratives (•, ★), ligatures PDF collées (fi -> fi), sauts de lignes inutiles et encodages Unicode non standard.

[CODE] Code source exact de `cleaner.py` :

```

def nettoyer_texte(texte_brut: str) -> str:
    if not texte_brut or not texte_brut.strip():
        return ""

    # 1. Normalisation Unicode NFC et correction des ligatures PDF
    texte = unicodedata.normalize("NFC", texte_brut)
    remplacements = {
        "\ufb01": "fi", "\ufb02": "fl", "\u2022": "", "\u00a0": " ", "\u2019": "'"
    }
    for original, rempl in remplacements.items():
        texte = texte.replace(original, rempl)

    # 2. Suppression des symboles parasites et en-têtes "Page X sur Y"
    for pattern in PATTERNS_BRUIT: # r"[|•••▶▶▶▶✓✓★☆@™°]"
        texte = re.sub(pattern, " ", texte, flags=re.MULTILINE)
    for pattern in PATTERNS_ENTETES: # r"Page\s*\d+\s*(sur|of|/)\s*\d+"
        texte = re.sub(pattern, "", texte, flags=re.MULTILINE | re.IGNORECASE)

    # 3. Correction des mots coupés par fin de ligne (ex: "Déve-\nloppueur" -> "Développeur")
    texte = re.sub(r"-\\n(\\w)", r"\\1", texte)

    # 4. Normalisation des espaces multiples et lignes vides
    texte = re.sub(r"[ \\t]+", " ", texte)
    texte = "\\n".join(ligne.strip() for ligne in texte.splitlines())
    return re.sub(r"\\n{3,}", "\\n\\n", texte).strip()

```

[EXPLICATION] Explication pour la soutenance :

- **Ligatures PDF** : Les ligatures comme `'\ufb01'` (fi) empêchent la recherche de mots-clés. Le dictionnaire les remplace par 'fi' propre pour que 'Profil' soit bien reconnu.

- **Correction des mots coupés** :L'expression régulière `re.sub(r'-(\w)', r'\1', texte)` rassemble les mots coupés par un tiret en fin de ligne dans les PDF.

3.3 Étape 3 : Parsing NLP et Motif Matching (ingestion/parser.py)

Cette étape extrait 3 informations clés du texte nettoyé : les compétences, les années d'expérience et le niveau de diplôme le plus élevé.

[CODE] Code source exact de parser.py :

```
# 1. Extraction des Compétences (croisement avec competences_ref.json)
def extraire_compétences(texte: str) -> List[str]:
    compétences_trouvees = set()
    zone_lower = texte.lower()
    for comp in COMPETENCES_REFERENCE:
        # Pattern avec limites de mot pour éviter de confondre C et C++ ou Java et
        # JavaScript
        pattern = r"(?![a-zA-Z0-9])" + re.escape(comp.lower()) + r"(?![a-zA-Z0-9])"
        if re.search(pattern, zone_lower):
            compétences_trouvees.add(comp)
    return sorted(compétences_trouvees)

# 2. Extraction du Niveau de Formation (Hiérarchie stricte)
FORMATIONS_PAR_NIVEAU = [
    ("Doctorat / PhD", [r"ph\.?d", r"doctorat", r"thèse"]),
    ("Master / Ingénieur", [r"master", r"ing[eé]nieur", r"msc", r"mohammadia",
    r"grande\s+[eé]cole"]),
    ("Licence / Bachelor", [r"licence", r"bachelor", r"b\.?sc\.?"]),
    ("BTS / DUT", [r"bts\b", r"dut\b", r"brevet\s+de\s+technicien"]),
    ("Baccalauréat", [r"baccalauréat", r"bac\b"])
]

def extraire_formation(texte: str) -> str:
    texte_lower = texte.lower()
    for niveau, patterns in FORMATIONS_PAR_NIVEAU:
        for pattern in patterns:
            if re.search(pattern, texte_lower):
                return niveau
    return "Non spécifié"
```

[EXPLICATION] Explication pour la soutenance :

- **Limites de mots (Word Boundaries)** :Le pattern `(?![a-zA-Z0-9])` Java `(?![a-zA-Z0-9])` garantit que le mot 'Java' ne match pas à l'intérieur de 'JavaScript'.
- **Hiérarchie des diplômes** :La liste `FORMATIONS_PAR_NIVEAU` est ordonnée du plus haut diplôme au plus bas. Si un CV contient 'Bac' ET 'Ingénieur', la boucle s'arrête dès qu'elle trouve 'Ingénieur', garantissant le diplôme le plus élevé.

3.4 Étape 4 : Orchestration et Assemblage JSON (ingestion/traiter_cv.py)

Le fichier `traiter_cv.py` assemble les 3 modules précédents et mesure la durée d'exécution.

[CODE] Code source exact de `traiter_cv.py` :

```

def traiter_cv(chemin_fichier: str) -> Dict[str, Any]:
    debut = time.perf_counter()
    nom_fichier = Path(chemin_fichier).name

    try:
        # Étape 1 : Extraction
        texte_extrait = extraire_texte(chemin_fichier)

        # Étape 2 : Nettoyage
        texte_propre = nettoyer_texte(texte_extrait)

        # Étape 3 : Parsing NLP
        competences = extraire_competences(texte_propre)
        experience_annees = extraire_experience_annees(texte_propre)
        formation = extraire_formation(texte_propre)

        # Étape 4 : Assemblage JSON
        duree = round(time.perf_counter() - debut, 3)
        return {
            "fichier": nom_fichier,
            "competences": competences,
            "experience_annees": experience_annees,
            "formation": formation,
            "texte_brut": texte_propre,
            "statut": "succès",
            "message": f"CV traité avec succès ({len(competences)} compétences).",
            "duree_traitement": duree
        }
    except Exception as e:
        return {"statut": "erreur", "message": str(e)}

```

IV. EXPLICATION DU FONCTIONNEMENT DES TESTS AUTOMATISÉS

Pour prouver la fiabilité du code au professeur, une suite de 21 tests automatisés dans tests/test_ingestion.py est exécutée.

4.1 Les 3 Groupes de Tests

- **1. TestCleaner (5 tests unitaires)** :Vérifient isolément que le nettoyeur élimine les symboles •, ★, corrige les ligatures PDF (fi -> fi) et supprime les espaces multiples.
- **2. TestParser (9 tests unitaires)** :Vérifient l'extraction des compétences (insensible à la casse), le calcul des durées d'expérience (mentions explicites vs calcul de dates) et la hiérarchie des diplômes.
- **3. TestIntegration (7 tests d'intégration)** :Génèrent dynamiquement de vrais fichiers PDF et DOCX temporaires et font passer tout le pipeline de A à Z.

4.2 Résultat de l'Exécution Pytest

```

===== test session starts =====
platform win32 -- Python 3.12.9, pytest-9.1.1
collected 21 items

```

V. VALIDATION EXPÉRIMENTALE SUR UN CV RÉEL

Le script `analyser_mon_cv.py` a été exécuté sur un vrai CV placé dans le répertoire `mes_cv/`. Résultats :

- ## 5.2 Fichier JSON Généré (mes_cv/MEKKI_FATIMA-EZAHRAE_AI_Backend_resultat.json)

VI. GUIDE DE SOUTENANCE : FOIRE AUX QUESTIONS (FAQ PROFESSEUR)

Voici les réponses aux questions techniques probables du professeur lors de la présentation :

- **Q1 : Comment le système gère-t-il les CV scannés ou sous forme d'image ?**Réponse : Le système utilise une architecture en cascade. Il tente d'abord l'extraction native textuelle rapide avec pdfplumber. Si le texte extrait fait moins de 80 caractères, le module bascule automatiquement sur PyMuPDF pour convertir la page en image HD (300 DPI), puis applique Tesseract OCR pour lire les caractères sur l'image.
- **Q2 : Comment évitez-vous de confondre 'Java' et 'JavaScript' lors de l'extraction des compétences ?**Réponse : Grâce aux 'Word Boundaries' (limites de mots en Regex). La recherche s'écrit `r'(?![a-zA-Z0-9])Java(?![a-zA-Z0-9])'`, ce qui exige que le mot soit entouré d'espaces ou de ponctuation, évitant de matcher à l'intérieur de 'JavaScript'.
- **Q3 : Comment calculez-vous les années d'expérience si le candidat ne l'écrit pas explicitement ?**Réponse : Si une mention explicite comme '5 ans d'expérience' n'est pas présente, le parseur extrait toutes les plages d'années (ex: 2020-2022 et 2022-2024), fusionne les périodes qui se chevauchent pour éviter les doubles comptages, et fait la somme du nombre total d'années d'activité.
- **Q4 : Si un CV mentionne plusieurs diplômes (Bac + Ingénieur), quel niveau est retenu ?**Réponse : La liste des diplômes est hiérarchisée du niveau le plus élevé (Doctorat) au plus bas (Bac). La boucle s'arrête au premier niveau trouvé. Ainsi, si un CV mentionne 'Bac en 2018' et 'Ingénieur en 2023', le niveau retenu est Master / Ingénieur.